

Dashboard Module — Component Library Architecture

What this is

An isolated, reusable **component library** for the Motadata ObserveOps **Dashboard module**, decomposed into **raw partials** — one folder per component with scoped `css` + a markup `html` fragment (+ optional `js` + `meta.json`). Part of the broader AIOps component library (sibling: `alerts_component/components/`).

Source & method

Decomposed from `dashboard_component/dashboard-module.html` — a single self-contained ~151 KB prototype that faithfully reproduces the live ObserveOps Dashboard module (`172.16.9.243/dashboard` , build 8.2.5), including its Add-Widget → Create-Widget flow, ~17 widget renderers, NOC wallboard, and AI Copilot.

Per-component extraction: read the component's slice of the monolith → ****decouple it from the monolith's global state**** (`WIDGETS` , `AI_INSIGHTS` , `META` , `renderCanvas` , hard DOM ids) → re-scope CSS under an `.oo-<name>` root → map every color to a real `var(--token)` → add hover/focus/ARIA ("active refinement") → adversarial verify (isolation, token discipline, render).

Folder structure

```
dashboard_component/components/
├─ tokens.css           dark+teal design tokens (+ radius/space/ring/on-accent)
├─ base.css            reset · webfonts · scrollbars · focus ring
├─ CONVENTIONS.md      the per-partial contract
├─ utils/              rng · series · esc · smooth-path · colors · spark (shared widget deps)
├─ icons/              card-icons · menu-icons · ui-icons (decoupled name→SVG maps)
├─ widgets/auto-mount.js the OOW renderer registry + declarative [data-widget] mounting
├─ build-gallery.mjs    scans partials → index.html (browsable demo + verification surface)
├─ build-extract-wave.mjs extraction pipeline + full component catalog (per-wave)
├─ gen-docs.mjs         generates docs/component-usage.md from every meta.json
├─ index.html           the generated gallery
├─ buttons/ fields/ items/ tiles/ widgets/ planes/
├   └─ <component>/ <name>.css · <name>.html · <name>.js? · meta.json
└─ docs/ architecture.md · color-palette.md · component-usage.md · README.md
```

Categories (82 components)

- **buttons/ (18)** — primary/ghost/outline, fab, icon-button, tabs, pills, range-pill, AI buttons...
- **fields/ (8)** — text/numeric/search/chat input, select, toggle, slider, checkbox.

- **items/ (12)** — tree node/leaf, badges, dots, chips, table-row, chat-message, ticker, menu-item...
- **tiles/ (8)** — widget-card, template/library cards, NOC tile/kpi, insight-card, metric-detail, empty-state.
- **widgets/ (17)** — chart, topn, gauge, pie, query, numeric, sankey, heatmap, stream, treemap, map,

anomaly, forecast, grid, alerts, events, ainsights. Each registers a **pure renderer** on the global

`registry ( 00w.<type> = (mountEl, opts) => {...} ),` depending only on `utils/`. Markup is a

declarative mount `<div data-widget="chart" data-seed="7"> ; widgets/auto-mount.js` wires them.

- **planes/ (19)** — shell, alert-banner, topbar, logo, subbar, sidebar, dashboard-panel, add-widget-panel,

create-widget-modal, overlay, canvas, ruler, noc-wallboard, ai-copilot-drawer, widget-fullscreen,

popup-menu, counter-picker, source-picker, toast. Composite surfaces that compose the atoms above.

Conventions (see `CONVENTIONS.md`)

CSS scoped under `.oo-<name>` , colors only via `var(--token)` , header comment, hover/focus/disabled states;

HTML a bare fragment with ARIA and comment-labelled variants (widgets use `data-widget` mounts); optional JS

as guarded document-level event delegation with **no monolith globals**; `meta.json` records

title/variants/states/deps/usedIn/tags. Quality gate enforces isolation, token discipline, a11y, render.

Build & verify

```
node build-gallery.mjs      # regenerate index.html, then screenshot with headless Chrome
node gen-docs.mjs          # regenerate the usage matrix from meta.json
# extraction waves run via the Workflow tool: edit ACTIVE=[categories] in build-extract-wave.mjs
```

Notes

- The source `dashboard-module.html` is preserved untouched.
- Widget renderers are pure functions of `opts, seed` (deterministic via the seeded `rng`), so the gallery

renders identically every load.